

Nghiên cứu phương pháp giải bài toán đếm theo chữ số

Gao Yihan

Trường Phổ thông trực thuộc Đại học Thanh Hoa

2009

Nguồn gốc: Nghiên cứu phương pháp giải bài toán đếm theo chữ số

IOI China candidate team papers / 2009 / Gao Yihan

Abstract

Bài toán đếm theo chữ số là một lớp bài khá phức tạp: độ khó thường không quá cao, nhưng nhiều trường hợp đặc biệt khiến việc cài đặt dễ rồi và dễ sai. Thông qua phân tích một vài ví dụ, bài viết trình bày một cách xử lý tốt hơn: xây dựng chương trình theo thứ tự từ đơn giản đến phức tạp, từng lớp một.

Từ khóa. Chữ số; bài toán đếm.

1 Mở đầu

Trong thi tin học có một lớp bài không quá khó nhưng rất rắc rối: bài toán đếm theo chữ số. Đặc điểm chính của chúng là đáp án của truy vấn liên quan tới các chữ số của mọi số trong một đoạn liên tiếp, đồng thời vẫn yêu cầu hiệu quả thời gian. Lời giải kiểu này thường kéo theo lượng mã lớn; nhiều trường hợp đặc biệt lại làm xác suất sai tăng cao. Vì vậy nhiều người không muốn động vào chúng. Nhưng nếu nắm một phương pháp tốt, loại bài này không khó như tưởng tượng.

2 Ví dụ 1: Tổng chữ số theo vị trí

Tóm tắt đề bài

Cho A, B , với $1 \leq A, B \leq 10^{15}$. Hãy tính tổng các chữ số của mọi số trong đoạn $[A, B]$ khi viết ở hệ cơ số k , với $2 \leq k \leq 10$.

Phân tích

Nếu có thể tính đáp án cho đoạn $[0, B]$, ta cũng tính được $[0, A - 1]$ và lấy hiệu để ra đáp án $[A, B]$. Vì vậy bài toán chuyển thành tính đáp án cho $[0, B]$.

Xét trường hợp đơn giản nhất: $B = k^n - 1$. Khi đó ta cần tính tổng chữ số của mọi số không quá n chữ số. Bổ sung số 0 ở đầu để mọi số đều có đúng n chữ số; khi ấy mỗi chữ số $0, 1, \dots, k - 1$ xuất hiện số lần bằng nhau. Do đó

$$ans = (B + 1)n \frac{0 + 1 + \dots + (k - 1)}{k} = \frac{(B + 1)n(k - 1)}{2}.$$

Ta có hàm đầu tiên:

```
long long getsum1(int n, int k)
// n là số chu số tự do, k là cơ số
{
    long long B = 1;
    for (int i = 0; i < n; i++) B *= k;
    return B * n * (k - 1) / 2;
}
```

Đồng thời nên viết hàm kiểm tra vết cạn:

```
long long check(int n, int k)
{
    long long ret = 0;
    for (int i = 1; i <= n; i++)
    {
        int t = i;
        while (t > 0)
        {
            ret += t % k;
            t /= k;
        }
    }
    return ret;
}
```

Việc viết chương trình kiểm tra cho từng khối là rất quan trọng trong các bài đếm theo chữ số, vì mỗi khối nhỏ thường dễ kiểm chứng bằng vết cạn.

Bây giờ quay lại bài toán gốc. Với một truy vấn, ta có thể dùng hàm trên xử lý mọi số có ít chữ số hơn B . Phần còn lại là các số có cùng số chữ số với B .

Nếu chữ số đầu đã nhỏ hơn chữ số đầu của cận trên, các chữ số còn lại tự do. Ví dụ trong hệ thập phân, để tính tổng chữ số của đoạn $[10000, 54321]$, ta phân rã thành:

$[10000, 19999], [20000, 29999], [30000, 39999], [40000, 49999],$
 $[50000, 54321].$

Bốn đoạn đầu có hậu tố tự do. Đoạn cuối lại tiếp tục phân rã:

$[50000, 50999], [51000, 51999], [52000, 52999], [53000, 53999],$
 $[54000, 54321],$

rồi tiếp tục tới khi chỉ còn đoạn rất nhỏ. Để xử lý trực tiếp một đoạn có tiền tố cố định, dùng:

```
long long getsum2(int prefixsum, int n, int k)
// prefixsum là tổng chu số tiền tố,
// n là độ dài phân tự do, k là cơ số
{
    long long B = getsum1(n, k);
```

```

    long long C = prefixsum;
    for (int i = 0; i < n; i++ ) C *= k;
    return B + C;
}

```

Sau đó dùng đệ quy để phân rã đoạn:

```

long long getsum3(int prefixsum, long long n, int k)
// tính tổng chu số của các số từ 0 đến n,
// với tiền tố có tổng chu số là prefixsum
{
    if ( n < k )
    {
        long long ret = 0;
        for (int i = 0; i <= n; i++ ) ret += prefixsum + i;
        return ret;
    }
    long long t = 1, tn = n;
    int d = 0;
    while ( tn >= k )
    {
        tn /= k;
        t *= k;
        d++;
    }
    long long ret = 0;
    for (int i = 0; i < tn; i++ )
        ret += getsum2(prefixsum + i, d, k);
    ret += getsum3(prefixsum + tn, n - t * tn, k);
    return ret;
}

```

Cuối cùng gọi `getsum3(0, n, k)` để tính đáp án.

Tiểu kết

Từ ví dụ đơn giản này, có thể thấy một phương pháp chung:

1. Trước hết xét trường hợp đơn giản nhất: chỉ cố định số chữ số, mọi chữ số còn lại tùy ý, và giải nó với độ phức tạp thấp.
2. Sau đó xét trường hợp có tiền tố cố định.
3. Cuối cùng chia bài toán gốc thành nhiều đoạn để dùng hai trường hợp trên.

Nếu một bài toán có thể phân rã thành tổng của các đoạn con như vậy, ta có thể xử lý bằng nguyên tắc trên. Nếu không, cần nghĩ cách biến đổi bài toán trước. Do bài đếm chữ số thường có chương trình kiểm tra rất dễ viết, nên nên chia lời giải thành nhiều khối nhỏ và kiểm tra từng khối riêng.

3 Ví dụ 2: The Sum

Tóm tắt đề bài

Viết các số từ 1 đến N , với $1 \leq N \leq 10^{15}$, lên giấy. Sau đó chèn luân phiên dấu “+” và “-” giữa các chữ số liền kề, rồi tính kết quả cuối cùng. Ví dụ khi $N = 12$:

$$+1 - 2 + 3 - 4 + 5 - 6 + 7 - 8 + 9 - 1 + 0 - 1 + 1 - 1 + 2 = 5.$$

Phân tích

Đây là bài đếm chữ số phức tạp hơn một chút. Theo nguyên tắc trên, trước hết xét trường hợp số chữ số cố định và mọi chữ số tự do.

Nếu số chữ số là chẵn, chẳng hạn 6 chữ số, dấu của từng vị trí là cố định:

$$+0 - 0 + 0 - 0 + 0 - 0, \quad +0 - 0 + 0 - 0 + 0 - 1, \quad \dots, \quad +9 - 9 + 9 - 9 + 9 - 9.$$

Mỗi vị trí đều có các chữ số 0 đến 9 xuất hiện đều nhau, nên tổng có thể tính trực tiếp.

Nếu số chữ số là lẻ, chẳng hạn 5 chữ số, các dòng liên tiếp triệt tiêu theo cặp, mỗi cặp có tổng -1 . Từ đó có hàm:

```
long long getsum1(int n, int k)
// n là số chữ số, k là tổng số chữ số, k >= n >= 1
{
    if ( k % 2 == 0 )
    {
        if ( n % 2 == 0 ) return 0;
        else
        {
            long long d = -45;
            for (int i = 0; i < n - 1; i++ ) d *= 10;
            return d;
        }
    }
    else
    {
        long long d = -1;
        for (int i = 0; i < n; i++ ) d *= 10;
        return d / 2;
    }
}
```

Hàm kiểm tra vẫn là vết cặn theo định nghĩa:

```
long long check(int n)
{
    long long ret = 0;
    int t = 1, a[10];
    for (int i = 1; i <= n; i++ )
```

```

{
    int r = 0, p = i;
    while ( p > 0 )
    {
        a[r++] = p % 10;
        p /= 10;
    }
    for (int j = r - 1; j >= 0; j-- )
    {
        ret += a[j] * t;
        t = -t;
    }
}
return ret;
}

```

Tiếp theo xét trường hợp có tiền tố. Vì tiền tố ảnh hưởng tới dấu của phần đuôi, hàm `getsum1` cần thêm tham số tổng số chữ số. Khi tổng số chữ số là chẵn, dấu của tiền tố không đổi và chỉ cần nhân với số dòng; khi tổng số chữ số là lẻ, các đóng góp của tiền tố triệt tiêu theo cặp.

```

long long getsum2(long long prefix, int n)
// prefix là tiền tố, n là số chữ số tự do
{
    int d = 0, t = 1;
    long long p = prefix, presum = 0;
    while ( p > 0 )
    {
        presum += (p % 10) * t;
        p /= 10;
        d++;
        t = -t;
    }
    presum *= -t;
    for (int i = 0; i < n; i++ ) presum *= 10;
    long long ret = getsum1(n, n + d);
    if ( (d + n) % 2 == 0 ) ret += presum;
    return ret;
}

```

Không thể dùng nguyên xi hàm đệ quy của ví dụ trước, vì trong bài này thêm số không ở đầu sẽ thay đổi kết quả. Tác giả dùng hàm phân rã riêng cho $[1, n]$; ý tưởng là xử lý trước mọi số ít chữ số hơn, rồi đi theo tiền tố của n . Phần mã chính:

```

long long getsum3(long long n)
// tính đáp án gốc trên [1, n]
{
    if ( n < 10 )
    {
        long long ret = 0;

```

```

        for (int i = 1; i <= n; i++ )
            if ( i % 2 == 0 ) ret -= i; else ret += i;
        return ret;
    }
    long long tn = n, p = 1;
    int d = 0;
    while ( tn > 0 ) { tn /= 10; d++; }
    for (int i = 1; i < d; i++ ) p *= 10;

    long long prefix = 0, ret = 5;
    for (int j = 1; j < d - 1; j++ )
        for (int i = 1; i <= 9; i++ )
            ret -= getsum2(i, j);

    tn = n;
    while (d > 1)
    {
        for (int i = 0; i < tn / p; i++ )
        {
            if ( prefix != 0 )
                ret -= getsum2(prefix, d - 1);
            prefix++;
        }
        tn %= p; p /= 10;
        d--; prefix *= 10;
    }
    /* Phan cuoi liet ke truc tiep cac so con lai. */
    return ret;
}

```

Bằng cách phân tích từng tầng từ đơn giản đến phức tạp, một bài toán vốn dễ rồi được biến thành các mô-đun rõ ràng. Mỗi mô-đun có chức năng xác định và có thể kiểm tra riêng, làm giảm xác suất sai. Dù tổng lượng mã tăng lên, thời gian suy nghĩ và thời gian sửa lỗi thường giảm.

4 Ví dụ 3: Graduated Lexicographical Ordering

Tóm tắt đề bài

Định nghĩa cách so sánh hai số như sau: trước hết so tổng chữ số; nếu khác nhau thì số có tổng chữ số lớn hơn được xem là lớn hơn; nếu tổng chữ số bằng nhau thì so theo thứ tự từ điển. Ví dụ:

$$120 < 4$$

vì tổng chữ số của 120 là 3, còn của 4 là 4;

$$555 < 78$$

vì theo thứ tự từ điển, “555” nhỏ hơn “78”; và

$$20 < 200$$

theo thứ tự từ điển. Bài toán yêu cầu tìm số nhỏ thứ k trong $1 \dots N$, và tìm vị trí của một số k trong thứ tự đó.

Phân tích

Bài có hai loại truy vấn. Loại thứ nhất, tìm số nhỏ thứ k , không trực tiếp áp dụng được khuôn mẫu trên. Với những truy vấn khó hợp nhất kết quả đoạn con, ta thường chuyển hóa bằng cách nhị phân đáp án hoặc xây đáp án theo từng chữ số.

Nếu thử một ứng viên C , ta có thể hỏi trong $1 \dots N$ có bao nhiêu số nhỏ hơn C . Nếu con số này là $K - 1$, thì C chính là đáp án cần tìm. Vì vậy phần lõi của bài là xử lý truy vấn thứ hai: vị trí của k trong $1 \dots N$.

Cấu trúc phân rã được mô tả như sau:

Truy vấn vị trí của k trong $1 \dots N$.

Tách thành: số lượng số có tổng chữ số nhỏ hơn $\text{sum}(k)$, và số lượng số có tổng chữ số bằng $\text{sum}(k)$ nhưng nhỏ hơn k theo thứ tự từ điển.

Các truy vấn phụ lại tách thành: số tổ hợp m chữ số có tổng bằng sum , số trong $1 \dots N$ có tổng chữ số bằng sum , và số có tổng chữ số bằng sum với tiền tố cố định.

Theo thứ tự từ đơn giản đến phức tạp, ta cài các hàm:

1. `getsum1`: hỏi số tổ hợp k chữ số có tổng chữ số bằng sum . Có thể dùng đệ quy và ghi nhớ để bảo đảm độ phức tạp.
2. `getsum2`: hỏi trong $1 \dots N$ có bao nhiêu số có tổng chữ số bằng sum . Cách làm tương tự hai ví dụ trước: chia đoạn theo tiền tố.
3. `getsum3`: hỏi trong $1 \dots N$ có bao nhiêu số có tổng chữ số bằng sum và có tiền tố prefix . Nếu prefix không còn là tiền tố của N , các chữ số sau không bị hạn chế và có thể dùng `getsum1`.
4. `getsum4`: hỏi trong $1 \dots N$ có bao nhiêu số có tổng chữ số bằng sum và nhỏ hơn k theo thứ tự từ điển. Ta phân loại theo tiền tố rồi cộng nhiều lần `getsum3`. Ví dụ với $k = 2423$, các tiền tố cần xét là 1, 20, 21, 22, 23, 240, 241, 2420, 2421, 2422.
5. `getsum5`: hỏi vị trí của k trong $1 \dots N$, bằng tổng của số lượng số có tổng chữ số nhỏ hơn $\text{sum}(k)$ và số lượng số có cùng tổng chữ số nhưng nhỏ hơn theo thứ tự từ điển.

Với truy vấn tìm số nhỏ thứ k , trong bài này không tiện nhị phân trực tiếp trên đáp án; có thể xác định đáp án bằng cách liệt kê tiền tố theo từng chữ số.

5 Tổng kết

Với bài toán đếm theo chữ số, bài viết đề xuất cách hiện thực từ đơn giản đến phức tạp: phân tích truy vấn, dần dần làm mịn cho tới khi gặp các trường hợp có thể xử lý trực tiếp. Khi cài đặt, thứ tự này làm cấu trúc mô-đun rõ ràng, dễ kiểm tra và gỡ lỗi. Nhiều hàm nhỏ liên hệ với nhau giúp mỗi khối mã không quá dài, giảm khả năng sai và giảm độ khó hiện thực.

Lời cảm ơn

Tác giả cảm ơn Shou Heming, biệt danh SHiningMoon, vì đã giúp đỡ.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.

Phụ lục

Đường dẫn bài gốc của ví dụ 2:

<https://www.spoj.pl/problems/KPSUM/>

Đường dẫn bài gốc của ví dụ 3:

<http://acm.zju.edu.cn/onlinejudge/showProblem.do?problemCode=2599>